

Quantization: A Practical Solution to Reduce the Size of Large Language Models (BERT, LLaMA, ...)

Prepared by: Hassan Gebrill

Abstract

Large pretrained models such as BERT and LLaMA deliver state-of-the-art performance but are expensive to store and run. Quantization compresses model weights and sometimes activations to lower-precision numeric types (e.g., 8-bit, 4-bit), drastically reducing memory footprint and improving inference throughput with limited accuracy loss. This document summarizes quantization types, trade-offs, popular tooling, and provides practical code examples for common workflows: (1) PyTorch dynamic quantization, (2) loading Transformer models in 8-bit via bit-sandbytes (Hugging Face integration), and (3) exporting and post-training quantizing models to ONNX for optimized inference.

1 Motivation

Modern transformer-based models often have hundreds of millions to hundreds of billions of parameters. This leads to:

- Large disk/storage costs.
- High GPU/CPU memory usage during inference and fine-tuning.
- Slower latency and throughput in production.

Quantization reduces the numerical precision of model parameters (and sometimes activations or optimizer states) to make models smaller and faster, enabling deployment on resource-limited hardware.

2 Types of Quantization

1. Post-Training Quantization (PTQ): Convert a trained model's weights (and possibly activations) from 32-bit float (FP32) to lower precision (FP16, INT8, INT4, or

other custom formats) without retraining. Quick but may reduce accuracy if the model is sensitive.

2. Quantization-Aware Training (QAT): Simulate quantization effects during training so the model learns to be robust to low precision. Higher accuracy than PTQ but requires extra training time.

3. Dynamic Quantization: Only weights are quantized ahead-of-time but activations are quantized/dequantized dynamically during inference. Useful for RNNs and transformer linear layers. Easy and often works well for language models.

4. Static (or Calibration) Quantization: Uses a representative dataset to compute activation ranges and quantizes both weights and activations with precomputed scales. Typically more accurate than naive PTQ.

5. Advanced formats and research: NF4 / 4-bit specialized schemes (used in some recent LLM quantization toolkits), mixed precision and block-wise quantization, and learned quantization transforms.

3 Trade-offs

- **Memory:** Lower precision reduces model size (e.g., FP32→INT8 gives 4x reduction).
- **Speed:** Reduced memory bandwidth and smaller models often result in faster inference.
- **Accuracy:** Aggressive quantization (4-bit or custom 4-bit schemes) can degrade accuracy unless careful methods (e.g., QAT or specialized quantizers) are used.
- **Hardware support:** Not all hardware or runtimes support all quantized formats. INT8 is widely supported; custom 4-bit formats might require specific runtimes (bitsandbytes, TVM, TensorRT, etc.).

4 Common Tooling

- **PyTorch native quantization:** dynamic/static/QAT APIs (`torch.quantization` / `torch.ao.quantization`).
- **bitsandbytes + transformers:** provides 8-bit and 4-bit loading for large transformers (integrates with Hugging Face ‘from_pretrained’ via ‘load_in_8bit=True’ etc.).
- **ONNX + ONNX Runtime:** export then run post-training quantization with onnxruntime quantization tools.

- **TensorRT / TFLite**: optimized inference with quantized models on supported hardware.

5 Practical Examples

Below are three practical examples you can copy into notebook cells.

5.1 Example 1 — PyTorch Dynamic Quantization (works well for many transformer linear layers)

This example shows applying dynamic quantization to a small transformer or LSTM-based classifier. Dynamic quantization is simple and often effective for CPU inference.

Listing 1: PyTorch dynamic quantization

```
1  # Install torch if needed:
2  # !pip install torch torchvision
3
4  import torch
5  from transformers import AutoModelForSequenceClassification,
   AutoTokenizer
6
7  model_name = "distilbert-base-uncased" # example small model
   for demo
8  tokenizer = AutoTokenizer.from_pretrained(model_name)
9  model = AutoModelForSequenceClassification.from_pretrained(
   model_name)
10
11 # Switch to eval
12 model.eval()
13
14 # Apply dynamic quantization (quantizes weights of Linear layers
   to qint8)
15 quantized_model = torch.quantization.quantize_dynamic(
16     model, {torch.nn.Linear}, dtype=torch.qint8
17 )
18
19 # Save and test
20 quantized_model.save_pretrained("distilbert_quantized_dynamic")
21 print("Saved quantized model.")
22
23 # Quick inference speed/size check
24 sample = "This is a test sentence."
```

```

25 inputs = tokenizer(sample, return_tensors="pt")
26 with torch.no_grad():
27     out = quantized_model(**inputs)
28 print("Output logits shape:", out.logits.shape)

```

Notes: Dynamic quantization is very easy and commonly used for CPU deployment. It quantizes weights into int8 but leaves activations in float and performs dequantize/quantize during execution.

5.2 Example 2 — Loading Transformers in 8-bit with bitsandbytes (Hugging Face integration)

This example shows how to load a large model in 8-bit using ‘bitsandbytes’ and ‘transformers’. This is a common approach to drastically reduce GPU memory when running large LLMs.

Listing 2: Hugging Face + bitsandbytes 8-bit loading

```

1  # Install dependencies (run once in your environment):
2  # !pip install transformers accelerate bitsandbytes
3
4  from transformers import AutoModelForCausalLM, AutoTokenizer
5  import torch
6
7  model_name = "gpt2" # replace with larger model when testing, e
   .g., "facebook/llama-7b" if available
8  tokenizer = AutoTokenizer.from_pretrained(model_name)
9
10 # Load model in 8-bit (requires bitsandbytes installed and a
   compatible transformers version)
11 model = AutoModelForCausalLM.from_pretrained(
12     model_name,
13     load_in_8bit=True, # enable 8-bit weights (
   bitsandbytes)
14     device_map="auto" # automatically place model on
   GPU(s)
15 )
16
17 # Example inference
18 input_text = "Once upon a time"
19 inputs = tokenizer(input_text, return_tensors="pt").to(model.
   device)
20 with torch.no_grad():
21     outputs = model.generate(**inputs, max_new_tokens=20)

```

```
22 print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Notes:

- `load_in_8bit = True` is a convenience that uses `bitsandbytes` under the hood. For very large models you may want to use `"auto"`.
- `Bitsandbytes` provides specialized kernels and quantization for efficient GPU memory use—this is crucial when loading multi-billion parameter models on limited GPUs.

5.3 Example 3 — Export to ONNX and Post-Training Quantization

Export a model to ONNX and use ONNX Runtime’s quantization tools to get an INT8 model optimized for CPU inference.

Listing 3: Export to ONNX and quantize with onnxruntime

```
1 # Install needed packages:
2 # !pip install onnx onnxruntime onnxruntime-tools transformers
3
4 from transformers import AutoTokenizer,
5     AutoModelForSequenceClassification
6 import torch
7 import onnx
8 from pathlib import Path
9 from onnxruntime.quantization import quantize_dynamic, QuantType
10
11 model_name = "distilbert-base-uncased"
12 tokenizer = AutoTokenizer.from_pretrained(model_name)
13 model = AutoModelForSequenceClassification.from_pretrained(
14     model_name)
15 model.eval()
16
17 # Dummy input
18 sample = "This is a quantization test."
19 inputs = tokenizer(sample, return_tensors="pt")
20
21 onnx_path = Path("distilbert.onnx")
22 torch.onnx.export(
23     model,
24     (inputs["input_ids"],),
25     str(onnx_path),
```

```
24     input_names=["input_ids"],
25     output_names=["logits"],
26     opset_version=13,
27     dynamic_axes={"input_ids": {0: "batch_size"}, "logits": {0:
        "batch_size"}}
28 )
29
30 # Post-training dynamic quantization (weights -> int8)
31 quantized_path = "distilbert_quantized.onnx"
32 quantize_dynamic(str(onnx_path), quantized_path, weight_type=
    QuantType.QInt8)
33 print("Saved quantized ONNX model:", quantized_path)
```

Notes:

- ONNX quantization is effective for CPU deployments and integrates with various runtimes.
- Static/ calibration-based quantization (using representative datasets) can often yield higher accuracy than dynamic quantization if activations need quantization.

6 Best Practices and Recommendations

- Start with **8-bit** quantization (INT8) where possible — it offers large savings with usually small accuracy loss.
- Use **dynamic quantization** for quick CPU-ready speedups and **QAT** if you need the best accuracy at low precision.
- For very large LLMs, combine **adapter methods** (LoRA) with quantized base weights (QLoRA-like approaches) — train only small adapters and keep base model quantized to reduce memory.
- Evaluate on downstream tasks and use calibration data for static quantization when quantizing activations.
- Check hardware and runtime support: some accelerators have native INT8 kernels; others perform better with BF16/FP16.

7 Limitations and Open Areas

- Aggressive quantization (4-bit) can require specialized quantizers and careful engineering (e.g., NF4, block-wise quantization).

- Not all libraries support training with custom 4-bit formats without additional tooling.
- Quantization of attention mechanisms and layernorm may need special handling.

8 Conclusion

Quantization is a powerful and practical technique to make large language models more usable in constrained environments. With multiple toolchains (PyTorch native quantization, bitsandbytes integration, ONNX quantization), practitioners can choose the best fit for their target hardware and accuracy constraints. Combining quantization with other parameter-efficient techniques (e.g., LoRA) yields further benefits for fine-tuning large models on limited hardware.

References and Further Reading

1. Jacob, B. et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference.
2. Nagel, M. et al. (2019). Data-free Quantization Through Weight Equalization and Bias Correction.
3. Dettmers, T. et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs.
4. PyTorch quantization docs: <https://pytorch.org/docs/stable/quantization.html>
5. Hugging Face + bitsandbytes documentation and examples.
6. ONNX Runtime quantization documentation.